

Assignment 02

Digital RTL Design

Combinational & Sequential Circuit Design

Marwan Abdelfattah

Contents

1	Question 1	2
1.1	N-Bit ALU with Parameterized Operation	2
1.1.1	N-Bit ALU	2
1.1.2	Randomized Self-Checking Testbenches	2
1.2	RTL Design	3
1.3	Testbench	4
1.3.1	Testbench-1 OPCODE = 0 (Addition)	4
1.3.2	Testbench-2 OPCODE = 1 (OR)	5
1.3.3	Testbench-3 OPCODE = 2 (Subtraction)	6
1.3.4	Testbench-4 OPCODE = 3 (XOR)	7
2	Question 2	8
2.1	Problem Statement	8
2.2	N-Bit Registered ALU with Synchronous Reset	8
2.2.1	Parameters	8
2.2.2	Ports	8
2.2.3	Design Requirements	8
2.3	RTL Design	9
3	Question 3	10
3.1	Data Latch with Active-Low Clear	10
3.2	RTL Design	10
4	Question 4	11
4.1	Parameterized Data Latch with Asynchronous Set and Clear	11
4.1.1	Parameters	11
4.1.2	Ports	11
4.2	RTL Design	11
5	Question 5	12
5.1	T-Type (Toggle) Flip-Flop with Active-Low Asynchronous Reset	12
5.1.1	T-FF RTL Design	12
5.2	Asynchronous D Flip-Flop with Active-Low Reset	13
5.2.1	D-FF RTL Design	13
5.3	Parameterized Asynchronous Flip-Flop	14
5.3.1	Inputs	14
5.3.2	Outputs	14
5.3.3	Parameter	14
5.3.4	D-FF Parameterized RTL Design	15

1 Question 1

1.1 N-Bit ALU with Parameterized Operation

Design the following circuit using Verilog and create randomized testbenches to verify its functionality.

1.1.1 N-Bit ALU

Implement an **N-bit ALU** according to the following specifications.

Parameters:

- **N**: Defines the width of the input and output ports. The default value is 4.
- **OPCODE**: Defines the operation to be performed. The default value is 0.

Ports:

- **In0**: Input with width N.
- **In1**: Input with width N.
- **Out**: Output with width N.

1.1.2 Randomized Self-Checking Testbenches

After implementing the ALU, create **four separate testbenches**. Each testbench should verify a different parameter configuration as follows:

- **Testbench 1**: $N = 4$, $OPCODE = 0$
- **Testbench 2**: $N = 4$, $OPCODE = 1$
- **Testbench 3**: $N = 4$, $OPCODE = 2$
- **Testbench 4**: $N = 4$, $OPCODE = 3$

Each testbench must satisfy the following requirements:

- The inputs **In0** and **In1** must be randomized.
- The testbench must be **self-checking**.
- The actual ALU output **Out** must be automatically compared with the expected output.
- The testbench should report whether each randomized test case passes or fails.

1.2 RTL Design

```
1 module N_bit_ALU(in0,in1,out);
2
3 parameter N = 4;
4 parameter OPCODE = 0;
5
6 input [N-1:0] in0, in1;
7 output [N-1:0] out;
8
9 assign out = (OPCODE==0)? (in0+in1) : // Addition
10              (OPCODE==1)? (in0|in1) : // OR
11              (OPCODE==2)? (in0-in1) : // Subtraction
12              (in0^in1); // XOR (OPCODE==3)
13
14 endmodule
```

1.3 Testbench

1.3.1 Testbench-1 OPCODE = 0 (Addition)

```
1 module alu_tb_opcode_0();
2
3 reg [3:0] in0, in1, out_expected;
4 wire [3:0] out;
5
6 N_bit_ALU #(.OPCODE(0)) DUT (.*);
7 integer i;
8
9 initial begin
10     for(i=0;i<20;i=i+1)begin
11         in0 = $random();
12         in1 = $random();
13         out_expected = in0 + in1;
14         #10
15         if(out_expected == out)begin
16             $display("Correct Output (Summation Process)");
17         end else begin
18             $display("OUTPUT FAILURE !!!!");
19             $stop;
20         end
21     end
22     $stop;
23 end
24
25 initial begin
26     $monitor("in0=%d, in1=%d, out_expected=%d, output=%d",
27             in0, in1, out_expected, out);
28 end
29
30 endmodule
```

1.3.2 Testbench-2 OPCODE = 1 (OR)

```
1 module alu_tb_opcode_1();
2
3 reg [3:0] in0, in1, out_expected;
4 wire [3:0] out;
5
6 N_bit_ALU #(.OPCODE(1)) DUT (.*);
7 integer i;
8
9 initial begin
10     for(i=0;i<20;i=i+1)begin
11         in0 = $random();
12         in1 = $random();
13         out_expected = in0 | in1;
14         #10
15         if(out_expected == out)begin
16             $display("Correct Output (OR Process)");
17         end else begin
18             $display("OUTPUT FAILURE !!!!");
19             $stop;
20         end
21     end
22     $stop;
23 end
24 initial begin
25     $monitor("in0=%b, in1=%b, out_expected=%b, output=%b",
26             in0, in1, out_expected, out);
27 end
28
29 endmodule
```

1.3.3 Testbench-3 OPCODE = 2 (Subtraction)

```
1 module alu_tb_opcode_2();
2
3 reg [3:0] in0, in1, out_expected;
4 wire [3:0] out;
5
6 N_bit_ALU #(.OPCODE(2)) DUT (.*);
7 integer i;
8
9 initial begin
10     for(i=0;i<20;i=i+1)begin
11         in0 = $random();
12         in1 = $random();
13         out_expected = in0 | in1;
14         #10
15         if(out_expected == out)begin
16             $display("Correct Output (OR Process)");
17         end else begin
18             $display("OUTPUT FAILURE !!!!");
19             $stop;
20         end
21     end
22     $stop;
23 end
24 initial begin
25     $monitor("in0=%b, in1=%b, out_expected=%b, output=%b",
26             in0, in1, out_expected, out);
27 end
28
29 endmodule
```

1.3.4 Testbench-4 OPCODE = 3 (XOR)

```
1 module alu_tb_opcode_3();
2
3 reg [3:0] in0, in1, out_expected;
4 wire [3:0] out;
5
6 N_bit_ALU #(.OPCODE(3)) DUT (.*);
7 integer i;
8
9 initial begin
10     for(i=0;i<20;i=i+1)begin
11         in0 = $random();
12         in1 = $random();
13         out_expected = in0 | in1;
14         #10
15         if(out_expected == out)begin
16             $display("Correct Output (OR Process)");
17         end else begin
18             $display("OUTPUT FAILURE !!!!");
19             $stop;
20         end
21     end
22     $stop;
23 end
24 initial begin
25     $monitor("in0=%b, in1=%b, out_expected=%b, output=%b",
26             in0, in1, out_expected, out);
27 end
28 endmodule
```

2 Question 2

2.1 Problem Statement

2.2 N-Bit Registered ALU with Synchronous Reset

Modify the design from Question 1 so that the output `Out` is registered and evaluated on the **positive edge of the clock**.

The ALU should retain the same functionality and parameterization as in Question 1, with the addition of a clock and an active-high synchronous reset.

2.2.1 Parameters

- `N`: Defines the width of the input and output ports. The default value is 4.
- `OPCODE`: Defines the operation to be performed. The default value is 0.

2.2.2 Ports

- `In0`: Input with width `N`.
- `In1`: Input with width `N`.
- `Out`: Registered output with width `N`.
- `Clk`: Clock input.
- `Rst`: Active-high synchronous reset.

2.2.3 Design Requirements

- The ALU operation should be evaluated on the **positive edge** of `Clk`.
- When `Rst` is HIGH at the positive edge of `Clk`, the output `Out` should be synchronously reset.
- When `Rst` is LOW, the result of the selected ALU operation should be stored in `Out` on the positive edge of `Clk`.
- The reset is **synchronous**, meaning that changes in `Rst` only affect `Out` on the positive edge of `Clk`.

The modified design should preserve the parameterized structure of the original ALU from Question 1 while adding the required sequential behavior to the output.

2.3 RTL Design

```
1 module N_bit_ALU(in0,in1,clk, rst, out);
2
3 parameter N = 4;
4 parameter OPCODE = 0;
5 input [N-1:0] in0, in1;
6 input clk, rst;
7 output reg [N-1:0] out;
8
9 always @(posedge clk) begin // Output is evaluated and updated only at the
   positive edge of clk
10     if(rst)begin
11         out <= 0;
12     end else
13         case (OPCODE)
14             2'b00: out <= in0 + in1;
15             2'b01: out <= in0 | in1;
16             2'b10: out <= in0 - in1;
17             2'b11: out <= in0 ^ in1;
18         endcase
19 end
20
21 endmodule
```

3 Question 3

3.1 Data Latch with Active-Low Clear

Implement a **data latch with an active-low clear input**. No testbench is required for this design.

- **Input:**

- **d**: Data input
- **gate**: Latch enable input
- **aclr**: Asynchronous active-low clear input

- **Output:**

- **q**: Data output

The latch should store the input data when **gate** is asserted. When **aclr** is asserted, the output **q** should be cleared to 0 asynchronously.

3.2 RTL Design

```
1 module data_latch(CLR, D, G, Q);
2
3 input CLR, D, G;
4 output reg Q;
5
6 always @(*) begin
7     if(~CLR) begin
8         Q <= 1'b0;
9     end
10    else if (G) begin
11        Q <= D;
12    end
13 end
14 endmodule
```

4 Question 4

4.1 Parameterized Data Latch with Asynchronous Set and Clear

Implement the following parameterized latch according to the given specifications. No testbench is required for this design.

4.1.1 Parameters

- **LAT_WIDTH**: Determines the width of the input data and output **q**.

4.1.2 Ports

- **aset**: Asynchronous set input. When asserted, it sets all bits of **q** to 1.
- **data**: Data input to the D-type latch with width **LAT_WIDTH**.
- **gate**: Latch enable input.
- **aclr**: Asynchronous clear input. When asserted, it sets all bits of **q** to 0.
- **q**: Data output from the latch with width **LAT_WIDTH**.

The latch should operate according to the following requirements:

- When **aclr** is asserted, **q** must be cleared to all zeros.
- When **aset** is asserted, **q** must be set to all ones.
- If both **aset** and **aclr** are asserted simultaneously, **aclr** must have priority over **aset**.
- When neither asynchronous control is asserted, the latch should store the input data when **gate** is enabled.

4.2 RTL Design

```
1 module latch(aset,data,gate,aclr,q);
2 parameter LAT_WIDTH = 4;
3
4 input aset, gate, aclr;
5 input [LAT_WIDTH-1:0] data;
6 output reg [LAT_WIDTH-1:0] q;
7
8 always @(*) begin
9     if(aclr)begin
10         q <= 0;
11     end else if (aset) begin
12         q <= {LAT_WIDTH{1'b1}};
13     end else if (gate) begin
14         q <= data;
15     end
16 end
17 endmodule
```

5 Question 5

5.1 T-Type (Toggle) Flip-Flop with Active-Low Asynchronous Reset

Implement a **T-type (toggle) flip-flop** with an active-low asynchronous reset. No testbench is required for this design.

The asynchronous reset should force **q** to 0 and **qbar** to 1.

- **Inputs:**

- **t**: Toggle input
- **rstn**: Active-low asynchronous reset
- **clk**: Clock input

- **Outputs:**

- **q**: Flip-flop output
- **qbar**: Complementary output

The T-flip-flop should operate as follows:

- When **rstn** is LOW, **q** must be set to 0 and **qbar** must be set to 1.
- When **rstn** is HIGH and **t** is HIGH, the outputs should toggle on the active clock edge.
- When **rstn** is HIGH and **t** is LOW, the output values should remain unchanged.

5.1.1 T-FF RTL Design

```
1 module T_FF (t, rstn, clk, q, qbar);
2
3 input t, rstn, clk;
4 output reg q, qbar;
5
6 always @(posedge clk or negedge rstn) begin
7     if (~rstn) begin
8         q <= 1'b0;
9         qbar <= 1'b1;
10    end else if (t) begin
11        q <= ~q;
12        qbar <= ~qbar;
13    end
14 end
15 endmodule
```

5.2 Asynchronous D Flip-Flop with Active-Low Reset

Implement an **asynchronous D-type flip-flop** with an active-low asynchronous reset. No testbench is required for this design.

The asynchronous reset should force **q** to 0 and **qbar** to 1.

- **Inputs:**

- **d**: Data input
- **rstn**: Active-low asynchronous reset
- **clk**: Clock input

- **Outputs:**

- **q**: Flip-flop output
- **qbar**: Complementary output

The flip-flop should capture the value of **d** on the active clock edge when **rstn** is HIGH. When **rstn** is LOW, the asynchronous reset must immediately force **q** to 0 and **qbar** to 1.

5.2.1 D-FF RTL Design

```
1 module D_FF (d, rstn, clk, q, qbar);
2     input d, rstn, clk;
3     output reg q, qbar;
4
5     always @(posedge clk or negedge rstn) begin
6         if (~rstn) begin
7             q <= 1'b0;
8             qbar <= 1'b1;
9         end else
10            q <= d;
11            qbar <= ~d;
12        end
13    endmodule
```

5.3 Parameterized Asynchronous Flip-Flop

Implement a **parameterized asynchronous flip-flop** with an active-low asynchronous reset according to the following specifications. No testbench is required for this design.

5.3.1 Inputs

- **d**: Data/toggle input
- **rstn**: Active-low asynchronous reset
- **clk**: Clock input

5.3.2 Outputs

- **q**: Flip-flop output
- **qbar**: Complementary output

5.3.3 Parameter

The module should contain a parameter called **FF_TYPE** that can take two valid values:

- **"DFF"**: Configure the design as a D-type flip-flop.
- **"TFF"**: Configure the design as a T-type flip-flop.

The default value of **FF_TYPE** should be:

FF_TYPE = "DFF"

The design should operate according to the selected value of **FF_TYPE**:

- When **FF_TYPE** = "DFF", the input **d** should act as the data input of the D flip-flop.
- When **FF_TYPE** = "TFF", the input **d** should act as the toggle input **t**.

For both configurations, **rstn** should function as an active-low asynchronous reset that forces:

$$q = 0$$

$$qbar = 1$$

The appropriate flip-flop behavior should be selected based on the value of the **FF_TYPE** parameter.

5.3.4 D-FF Parameterized RTL Design

```
1 module D_FF (d, rstn, clk, q, qbar);
2     parameter FF_TYPE = "DFF";
3     input d, rstn, clk;
4     output reg q, qbar;
5
6     always @(posedge clk or negedge rstn) begin
7         if (~rstn) begin
8             q <= 1'b0;
9             qbar <= 1'b1;
10
11         end else begin
12             if (FF_TYPE == "DFF") begin
13                 q <= d;
14                 qbar <= ~d;
15             end
16             else if (FF_TYPE == "TFF") begin
17                 if(d) begin
18                     q <= ~q;
19                     qbar <= ~qbar;
20                 end
21             end
22         end
23
24     end
25 endmodule
```